Machine Coding: Search and Filter Product List

- **Problem:** Show a product list that the user can narrow down with a text search (name or brand), category, minimum rating, a price range and an "in stock only" toggle. Filters combine, the result count updates live, there is a reset, and the filtered view survives a refresh / can be shared through the URL. Products load from a (mock) API with loading and error states.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Assumptions written down
Use cases + architecture + state	8 min	Table, component tree, filters object
Core build (load list, search, category)	17 min	Working list with 2 filters
Rating, price range, in-stock, reset, count	12 min	All filters combined
Loading/error/empty states, URL sync, a11y	10 min	Polished behaviour
Test manually + wrap-up	8 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"I'll keep all filters in one object and write one pure* `filterProducts` *function. Then the UI only renders, and I can test the filtering without React."* If you run out of time, a clear plan you talked through still scores well.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Load products from an API (mocked) and show them as cards.
- Search by product name **or** brand: live, case-insensitive, ignoring surrounding spaces.
- Filter by category (options come from the data), minimum rating, price range (min / max) and "in stock only".
- All filters combine (AND).
- Show "Showing X of Y products", a Reset button, and a clear empty state.
- Keep filters in the URL so refresh and shared links show the same view.

Non-functional requirements

Concern	What it means here
Performance	Filtering 10-1,000 items in memory on each keystroke is instant. Debounce / `useMemo` only when it's measured as slow, or when search calls an API.
Scalability	One filters object + one predicate per filter: a new filter is one field + one line.
Error handling	API failure shows a message + Retry; invalid price range shows an error instead of an empty list; garbage values in the URL fall back to defaults.
Accessibility	Every control has a visible label (or `aria-label`), price inputs are grouped in a `fieldset`, `role="search"` on the form, `aria-live` on the count, `role="alert"` on errors.
Reusability	`filterProducts`, `parsePriceRange` and the URL helpers are pure functions, independent of React.
Extensibility	Server-side filtering later = send the same filters object as query params.

Clarifying questions to ask the interviewer

1. Where does the data come from (static array or API), and how many products: tens, thousands, or millions?
2. Should search match only the name, or also the brand / category?
3. Do filters combine with AND? Is category single-select or multi-select?
4. How should the price filter work: dropdowns, two inputs, or a slider? What if min > max?
5. Should the filters survive a refresh or be shareable by URL?
6. Is sorting (price, rating) in scope? (Usually: mention it as an extension.)

Assumptions we will build with

- Data comes from a mocked async API (600 ms delay), about 10-1,000 products, filtered on the client.
- Search matches name or brand, case-insensitive, trimmed.
- Single-select category; the options are built from the data (no hard-coded list).
- Rating means "this rating and above" (`4` shows 4.0-5.0).

- Price: two optional number inputs. Empty means no limit. Negative values or min > max show an error, and the price filter is ignored until the user fixes it.
- Filters sync to the URL with `replaceState` (no new history entry per keystroke).

2. Identify interactions / use cases

Action / input	Expected behavior / output
Open the page	"Loading products..." then the cards and "Showing 10 of 10 products"
API fails	Error message + Retry button; Retry shows loading again and refetches
Type "dell" in search	Only Dell products; count updates live
Type " LAPTOP "	Same as "laptop" (trimmed, case-insensitive)
Pick category "Appliances"	Only appliances. The option exists because categories come from the data
Pick rating "4★ & up"	Only products rated 4.0 or higher
Min 1000, Max 30000	Products priced 1,000-30,000 inclusive
Only Min set	No upper limit
Min 40000, Max 1000	Error "Min price cannot be more than max price"; price filter ignored
Negative price	Error "Prices must be positive numbers"
Tick "In stock only"	Sold-out products hidden
Combine search + category + rating + price	All rules apply together (AND)
No product matches	"No products match your filters." + "Clear all filters"
Click Reset	All filters back to defaults; Reset disabled when nothing is active
Change any filter	URL updates, e.g. `?q=dell&category=Electronics&rating=4`
Open a shared URL / refresh	Same filters pre-filled, same results
URL has `?rating=abc`	Falls back to the default ("Any")

3. Design the architecture

Module	Responsibility
`App`	Owns products + request status + filters; derives the visible list, categories, price error; syncs the URL
`api/products.js`	`fetchProducts()` : mock API (delay, optional failure)
`utils/filters.js`	Pure logic: `filterProducts` , `parsePriceRange` , `getCategories` , `filtersFromURL` / `filtersToURL` , `DEFAULT_FILTERS`
`FilterBar`	Stateless form: search, category, rating, price, in-stock, reset
`ProductCard`	Shows one product (name, brand, category, price, rating, sold-out badge)

```
App  (products, status, error, attempt, filters)
├── FilterBar        ← filters, categories, priceError → onChange(name, value), onReset
├── status message   (loading / error + Retry)
├── count            "Showing X of Y"
└── <ul> visibleProducts.map
     └── ProductCard ← product
utils/filters.js  ← pure functions used by App
api/products.js   ← mock fetch used by App
```

Start simple. First version: hard-coded array + one search input + `.filter()` in `App`. Move the logic into `utils/filters.js` once there are 3+ filters, and add the API layer when loading/error states come in.

4. Design the state / data model

State	Purpose
`products`	Raw list from the API (never mutated or filtered in place)
`status`	`loading | success | error`: one enum instead of `isLoading` + `isError` booleans
`error`	Error message to show
`attempt`	Retry counter; changing it re-runs the fetch effect
`filters`	`{ search, category, minRating, minPrice, maxPrice, inStock }` in one object

```
export const DEFAULT_FILTERS = { search: "", category: "all", minRating: 0, minPrice: "", maxPrice: "",
inStock: false };

const [products, setProducts] = useState([]);
const [status, setStatus] = useState("loading");
const [error, setError] = useState("");
const [attempt, setAttempt] = useState(0);
const [filters, setFilters] = useState(() => filtersFromURL(window.location.search)); // lazy: read URL
once

const updateFilter = (name, value) => setFilters((prev) => ({ ...prev, [name]: value }));

// derived
const categories = getCategories(products);
const visibleProducts = filterProducts(products, filters);
const priceError = parsePriceRange(filters).error;
const hasActiveFilters = JSON.stringify(filters) !== JSON.stringify(DEFAULT_FILTERS);
```

Senior note

- **Derived, not stored:** `visibleProducts`, `categories`, `priceError`, `hasActiveFilters` and the count. Storing `filteredProducts` in state (and updating it in every handler) is the classic bug source: you

change one filter and forget to apply the others.
- **One filters object** instead of 6 `useState`s: one `updateFilter(name, value)` handler, one-line reset (`setFilters(DEFAULT_FILTERS)`), and easy URL serialisation.
- **Price stays a string** (what the input gives). Parse it in one place (`parsePriceRange`), so an empty input is "no limit", not `0`.
- **Ownership:** everything lives in `App`; `FilterBar` and `ProductCard` are stateless. The URL is a *mirror* of `filters` (read once on load, then written), not a second source of truth.

5. Design the data structures and contracts

Product (stable numeric `id` from the backend, used as the React key):

```
{ "id": 106, "name": "Refrigerator", "category": "Appliances", "price": 35000, "status": "available",
"brand": "LG", "rating": 4.6 }
```

`price` is a number in rupees; `status` is `"available" | "sold out"`; `rating` is 0-5 with one decimal.

API contract (mocked)

Call	Input	Success	Failure
`fetchProducts()` ≈ `GET /api/products`	-	`Product[]` after ~600 ms	rejects with `Error("Could not load products. Please try again.")`

With server-side filtering, the same filters object becomes the query string: `GET /api/products?q=dell&category=Electronics&rating=4&min=1000&max=60000&stock=1`.

Filter contract: `filterProducts(products, filters) → Product[]` (pure, keeps the original order). `parsePriceRange({ minPrice, maxPrice }) → { min, max, error }`.

URL contract: `q`, `category`, `rating`, `min`, `max`, `stock=1`. Default values are left out, so the clean URL is just `/`.

6. Trace the data flow

User types "dell" with "In stock only" already ticked:

```
User types "d", "e", "l", "l" in the search input
  ↓ FilterBar: onChange("search", "dell") on each keystroke
  ↓ App: setFilters(prev => ({ ...prev, search: "dell" }))
  ↓ React re-renders App
  ↓ derived: filterProducts(products, filters)
  ↓   query = "dell" → name OR brand includes it, AND inStock → only available products
  ↓ effect: history.replaceState(…, "?q=dell&stock=1")
  ↓ UI: one card (Laptop · Dell), "Showing 1 of 10 products" (announced by aria-live)
```

7. Important technical challenges

Challenge 1: combining many filters without bugs

- *Problem:* Using separate state per filter and chained `.filter()` calls with "magic" values breaks easily. In the first version, the state starts as `"All"` but the rating option's value is `"all"`. Choosing "All" again sets `"all"`, `Number("all")` is `NaN`, and **every product disappears**.
- *Solution:* One filters object with typed defaults (`minRating: 0`, `category: "all"`), and one pure predicate where each line is one rule.

```
return products.filter((p) =>
  (!query || p.name.toLowerCase().includes(query) || p.brand.toLowerCase().includes(query)) &&
  (filters.category === "all" || p.category === filters.category) &&
  p.rating >= filters.minRating &&
  (min === null || p.price >= min) &&
  (max === null || p.price <= max) &&
  (!filters.inStock || p.status === "available"));
```

Challenge 2: filter options that match the data

- *Problem:* Hard-coded categories go stale. The data has "Appliances" but the dropdown doesn't, so the Refrigerator can never be selected by category.
- *Solution:* Derive the options from the products.

```
export const getCategories = (products) => [...new Set(products.map((p) => p.category))].sort();
```

Challenge 3: price range validation

- *Problem:* The user types Min 40000 and Max 1000. A naive filter shows 0 products with no reason given. An empty input becoming `Number("") === 0` silently means "max price 0".
- *Solution:* Keep the inputs as strings, parse them in one function, treat "" as no limit, and return an error instead of a broken filter.

```
const min = minPrice === "" ? null : Number(minPrice);
const max = maxPrice === "" ? null : Number(maxPrice);
if (min !== null && max !== null && min > max)
  return { min: null, max: null, error: "Min price cannot be more than max price" };
```

Challenge 4: async loading + URL as shareable state

- *Problem:* (a) If the user clicks Retry twice, or the component unmounts mid-request, a late response can overwrite newer state. (b) A refresh wipes the filters; a user can't send a filtered link to a friend.
- *Solution:* Ignore stale responses with a cleanup flag. Read the filters from the URL once (lazy initialiser) and mirror them back with `replaceState`, so each keystroke doesn't add a history entry.

```
useEffect(() => {
  let ignore = false;
```

```
  fetchProducts().then((data) => { if (!ignore) { setProducts(data); setStatus("success"); } })
                 .catch((err) => { if (!ignore) { setError(err.message); setStatus("error"); } });
  return () => { ignore = true; };
}, [attempt]);

useEffect(() => { window.history.replaceState(null, "", filtersToURL(filters)); }, [filters]);
```

8. Implement in the correct order

#	Step	Working result
1	Hard-coded products rendered as cards with `key={p.id}`	List visible
2	Search input (name only) + derived list	Search works
3	Filters object + `updateFilter`; add brand to search	One state shape for everything
4	Category select built from the data	Category filter works
5	Rating select (`minRating` number)	Rating filter works
6	Price min/max inputs + `parsePriceRange` + error	Price filter works safely
7	In-stock checkbox	All filters combine
8	Move logic to `utils/filters.js`	Pure, testable logic
9	Count + empty state + Reset / Clear all	Clear feedback
10	Mock `fetchProducts` + loading / error / Retry with the stale-response guard	Async states done
11	URL sync (`filtersFromURL` + `replaceState`)	Refresh / share works
12	Labels, `fieldset`, `aria-live`, `role="alert"`, responsive grid	Polished + accessible

9. Cover all states and edge cases

State / edge case	Handling
Initial / loading	"Loading products..." (`role="status"`); filters already filled in from the URL
Success	Cards in a responsive grid + "Showing X of Y products"
Error	Message + Retry (`role="alert"`); Retry goes back to loading
Empty result	"No products match your filters." + "Clear all filters"
Empty data from the API	"Showing 0 of 0 products" + the same empty state
Search with spaces / mixed case	Trimmed and lowercased before matching
Search matches brand only	Still shown (name OR brand)
Category not in the data (old link)	0 results + clear-filters button (no crash)

State / edge case	Handling
Rating boundary (exactly 4.0)	Included (`>=`)
Price boundary (exactly min/max)	Included (inclusive range)
Only min or only max	The other side has no limit
Min > max / negative price	Inline error; price filter ignored until it's fixed
Non-numeric `rating` in the URL	Falls back to 0 ("Any")
Nothing filtered	Reset disabled; URL is clean `/`
Retry clicked twice / unmount during fetch	Stale response ignored by the cleanup flag
Sold-out product	Dimmed card + "Sold out" badge; hidden by "In stock only"
Narrow screen	Filters wrap; the grid becomes 1 column; no horizontal scroll

10. Final structure

```
_03_Search_and_Filter_List/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css              ← global reset
    ├── App.jsx                ← state owner: products, status, filters; derived list; URL sync
    ├── App.css                ← layout, filter bar, grid, cards
    ├── api/
    │   └── products.js        ← mock GET /api/products
    ├── utils/
    │   └── filters.js         ← filterProducts, parsePriceRange, getCategories, URL helpers
    └── components/
        ├── FilterBar.jsx      ← stateless filter form
        └── ProductCard.jsx    ← one product
```

11. Full working code

`src/api/products.js`

```
const PRODUCTS = [
  { id: 101, name: "Laptop", category: "Electronics", price: 50000, status: "available", brand: "Dell",
rating: 4.5 },
  { id: 102, name: "Smartphone", category: "Electronics", price: 20000, status: "sold out", brand:
"Samsung", rating: 4.2 },
  { id: 103, name: "T-Shirt", category: "Clothing", price: 500, status: "available", brand: "Levis",
rating: 4.0 },
  { id: 104, name: "Coffee Mug", category: "Kitchen", price: 300, status: "available", brand:
```

```
"Crockery", rating: 4.1 },
  { id: 105, name: "Headphones", category: "Electronics", price: 1500, status: "available", brand:
"Sony", rating: 4.3 },
  { id: 106, name: "Refrigerator", category: "Appliances", price: 35000, status: "available", brand:
"LG", rating: 4.6 },
  { id: 107, name: "Jeans", category: "Clothing", price: 1200, status: "sold out", brand: "Wrangler",
rating: 4.0 },
  { id: 108, name: "Blender", category: "Kitchen", price: 2000, status: "available", brand: "Philips",
rating: 4.2 },
  { id: 109, name: "Tablet", category: "Electronics", price: 25000, status: "available", brand: "Apple",
rating: 4.7 },
  { id: 110, name: "Sneakers", category: "Footwear", price: 2500, status: "available", brand: "Nike",
rating: 4.4 },
];

// shortcut: in-memory mock of GET /api/products; replace with fetch() when a real API exists.
export async function fetchProducts({ fail = false } = {}) {
  await new Promise((resolve) => setTimeout(resolve, 600));
  if (fail) throw new Error("Could not load products. Please try again.");
  return PRODUCTS;
}
```

src/utils/filters.js

```
export const DEFAULT_FILTERS = { search: "", category: "all", minRating: 0, minPrice: "", maxPrice: "",
inStock: false };

// Price inputs are strings: "" means "no bound". Returns { min, max, error }.
export function parsePriceRange({ minPrice, maxPrice }) {
  const min = minPrice === "" ? null : Number(minPrice);
  const max = maxPrice === "" ? null : Number(maxPrice);
  if ((min !== null && (Number.isNaN(min) || min < 0)) || (max !== null && (Number.isNaN(max) || max <
0)))
    return { min: null, max: null, error: "Prices must be positive numbers" };
  if (min !== null && max !== null && min > max)
    return { min: null, max: null, error: "Min price cannot be more than max price" };
  return { min, max, error: "" };
}

// One pure function, one rule per filter: easy to read, test and extend.
export function filterProducts(products, filters) {
  const query = filters.search.trim().toLowerCase();
  const { min, max } = parsePriceRange(filters); // an invalid range is ignored, the UI shows its error
  return products.filter(
    (p) =>
      (!query || p.name.toLowerCase().includes(query) || p.brand.toLowerCase().includes(query)) &&
      (filters.category === "all" || p.category === filters.category) &&
```

```
      p.rating >= filters.minRating &&
      (min === null || p.price >= min) &&
      (max === null || p.price <= max) &&
      (!filters.inStock || p.status === "available")
  );
}

// Categories come from the data, so a new category never needs a code change.
export function getCategories(products) {
  return [...new Set(products.map((p) => p.category))].sort();
}

// URL <-> filters, so a filtered view survives refresh and can be shared.
export function filtersFromURL(search) {
  const params = new URLSearchParams(search);
  const rating = Number(params.get("rating"));
  return {
    search: params.get("q") ?? "",
    category: params.get("category") ?? "all",
    minRating: rating >= 0 && rating <= 5 ? rating : 0, // garbage in the URL falls back to the default
    minPrice: params.get("min") ?? "",
    maxPrice: params.get("max") ?? "",
    inStock: params.get("stock") === "1",
  };
}

export function filtersToURL(filters) {
  const params = new URLSearchParams();
  if (filters.search) params.set("q", filters.search);
  if (filters.category !== "all") params.set("category", filters.category);
  if (filters.minRating) params.set("rating", String(filters.minRating));
  if (filters.minPrice) params.set("min", filters.minPrice);
  if (filters.maxPrice) params.set("max", filters.maxPrice);
  if (filters.inStock) params.set("stock", "1");
  const query = params.toString();
  return query ? `?${query}` : window.location.pathname;
}
```

src/App.jsx

```
import { useEffect, useState } from "react";
import "./App.css";
import { fetchProducts } from "./api/products";
import {
  DEFAULT_FILTERS,
  filterProducts,
  filtersFromURL,
```

```
  filtersToURL,
  getCategories,
  parsePriceRange,
} from "./utils/filters";
import FilterBar from "./components/FilterBar";
import ProductCard from "./components/ProductCard";

const FAIL_DEMO = new URLSearchParams(window.location.search).has("fail"); // add ?fail to see the error
state

function App() {
  const [products, setProducts] = useState([]);
  const [status, setStatus] = useState("loading"); // loading | success | error
  const [error, setError] = useState("");
  const [attempt, setAttempt] = useState(0);
  const [filters, setFilters] = useState(() => filtersFromURL(window.location.search));

  useEffect(() => {
    let ignore = false; // drop the response if a newer request started or the component unmounted
    fetchProducts({ fail: FAIL_DEMO && attempt === 0 })
      .then((data) => {
        if (ignore) return;
        setProducts(data);
        setStatus("success");
      })
      .catch((err) => {
        if (ignore) return;
        setError(err.message);
        setStatus("error");
      });
    return () => {
      ignore = true;
    };
  }, [attempt]);

  useEffect(() => {
    window.history.replaceState(null, "", filtersToURL(filters));
  }, [filters]);

  // Derived, never stored.
  const categories = getCategories(products);
  const visibleProducts = filterProducts(products, filters);
  const priceError = parsePriceRange(filters).error;
  const hasActiveFilters = JSON.stringify(filters) !== JSON.stringify(DEFAULT_FILTERS);

  const updateFilter = (name, value) => setFilters((prev) => ({ ...prev, [name]: value }));
  const resetFilters = () => setFilters(DEFAULT_FILTERS);
```

```
const retry = () => {
  setStatus("loading");
  setAttempt((a) => a + 1);
};

return (
  <main className="container">
    <h1 className="title">Search and Filter</h1>

    <FilterBar
      filters={filters}
      categories={categories}
      priceError={priceError}
      onChange={updateFilter}
      onReset={resetFilters}
      hasActiveFilters={hasActiveFilters}
    />

    {status === "loading" && <p className="message" role="status">Loading products...</p>}

    {status === "error" && (
      <div className="message" role="alert">
        <p>{error}</p>
        <button type="button" onClick={retry}>
          Retry
        </button>
      </div>
    )}

    {status === "success" && (
      <>
        <p className="count" aria-live="polite">
          Showing {visibleProducts.length} of {products.length} products
        </p>
        {visibleProducts.length === 0 ? (
          <div className="message">
            <p>No products match your filters.</p>
            <button type="button" onClick={resetFilters}>
              Clear all filters
            </button>
          </div>
        ) : (
          <ul className="products">
            {visibleProducts.map((p) => (
              <ProductCard key={p.id} product={p} />
            ))}
          </ul>
```

```
          )}
        </>
      )}
    </main>
  );
}

export default App;
```

src/components/FilterBar.jsx

```
const RATINGS = [0, 1, 2, 3, 4];

// Stateless: every value comes from props, every change goes up through onChange(name, value).
const FilterBar = ({ filters, categories, priceError, onChange, onReset, hasActiveFilters }) => (
  <form className="filter-bar" onSubmit={(e) => e.preventDefault()} role="search">
    <label className="field grow">
      <span>Search</span>
      <input
        type="search"
        placeholder="Search by name or brand"
        value={filters.search}
        onChange={(e) => onChange("search", e.target.value)}
      />
    </label>

    <label className="field">
      <span>Category</span>
      <select value={filters.category} onChange={(e) => onChange("category", e.target.value)}>
        <option value="all">All</option>
        {categories.map((c) => (
          <option key={c} value={c}>
            {c}
          </option>
        ))}
      </select>
    </label>

    <label className="field">
      <span>Rating</span>
      <select value={filters.minRating} onChange={(e) => onChange("minRating", Number(e.target.value))}>
        {RATINGS.map((r) => (
          <option key={r} value={r}>
            {r === 0 ? "Any" : `${r}★ & up`}
          </option>
        ))}
      </select>
```

```
    </label>

    <fieldset className="field price">
      <legend>Price (₹)</legend>
      <input
        type="number"
        min="0"
        placeholder="Min"
        aria-label="Minimum price"
        value={filters.minPrice}
        onChange={(e) => onChange("minPrice", e.target.value)}
        aria-invalid={Boolean(priceError)}
        aria-describedby={priceError ? "price-error" : undefined}
      />
      <input
        type="number"
        min="0"
        placeholder="Max"
        aria-label="Maximum price"
        value={filters.maxPrice}
        onChange={(e) => onChange("maxPrice", e.target.value)}
        aria-invalid={Boolean(priceError)}
        aria-describedby={priceError ? "price-error" : undefined}
      />
    </fieldset>

    <label className="checkbox">
      <input type="checkbox" checked={filters.inStock} onChange={(e) => onChange("inStock",
e.target.checked)} />
      In stock only
    </label>

    <button type="button" onClick={onReset} disabled={!hasActiveFilters}>
      Reset
    </button>

    {priceError && (
      <p id="price-error" role="alert" className="error">
        {priceError}
      </p>
    )}
  </form>
);

export default FilterBar;
```

src/components/ProductCard.jsx

```
const ProductCard = ({ product }) => {
  const soldOut = product.status !== "available";
  return (
    <li className={`product-card ${soldOut ? "sold-out" : ""}`}>
      <h3>{product.name}</h3>
      <p className="brand">{product.brand}</p>
      <p>{product.category}</p>
      <p className="price">₹{product.price.toLocaleString("en-IN")}</p>
      <p aria-label={`Rated ${product.rating} out of 5`}>★ {product.rating.toFixed(1)}</p>
      {soldOut && <span className="badge">Sold out</span>}
    </li>
  );
};

export default ProductCard;
```

src/App.css

```
body { background: gainsboro; font-family: system-ui, sans-serif; }
.container { max-width: 1000px; margin: 0 auto; padding: 16px; background: white; border-radius: 8px; }
.title { text-align: center; margin-bottom: 16px; }

.filter-bar { display: flex; flex-wrap: wrap; gap: 12px; align-items: flex-end; margin-bottom: 12px; }
.field { display: flex; flex-direction: column; gap: 4px; font-size: 14px; font-weight: 600; border: none; }
.field.grow { flex: 1 1 220px; }
.field.price { flex-direction: row; gap: 6px; }
.field.price legend { margin-bottom: 4px; }
.field.price input { width: 90px; }
input[type="search"], input[type="number"], select { padding: 8px 10px; border: 1px solid #ccc; border-radius: 6px; font-size: 15px; }
input[aria-invalid="true"] { border-color: #dc2626; }
.checkbox { display: flex; gap: 6px; align-items: center; padding-bottom: 8px; }
button { padding: 8px 14px; border: none; border-radius: 6px; background: #4f46e5; color: white; cursor: pointer; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
.error { width: 100%; color: #dc2626; font-size: 14px; }

.count { color: #4b5563; margin-bottom: 8px; }
.message { text-align: center; padding: 32px; color: #4b5563; display: flex; flex-direction: column; gap: 12px; align-items: center; }

.products { list-style: none; display: grid; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); gap: 12px; }
.product-card { position: relative; padding: 14px; border: 1px solid #f0b84d; border-radius: 10px; box-shadow: 0 2px 6px rgba(0, 0, 0, 0.08); }
```

```
.product-card h3 { font-size: 1.05rem; margin-bottom: 4px; }
.product-card p { font-size: 0.9rem; color: #555; margin: 2px 0; }
.product-card .price { font-weight: bold; color: #e65100; }
.product-card.sold-out { opacity: 0.6; }
.badge { position: absolute; top: 10px; right: 10px; background: #dc2626; color: white; font-size: 12px;
padding: 2px 6px; border-radius: 4px; }
```

src/index.css

```
*{
  margin:0;
  padding:0;
  box-sizing:border-box;
  --webkit-font-smoothing:antialiased;
  text-rendering: optimizeLegibility;
}

body{
  padding:10px;
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: trimmed and case-insensitive search across name and brand, categories built from the data, inclusive rating/price boundaries, one-sided price ranges, min > max and negative prices, in-stock toggle, empty data, empty results with a clear-all button, API error with Retry, stale responses ignored, invalid URL values, responsive layout.

Trade-offs to mention to the interviewer

- *Client-side vs server-side filtering:* client-side is instant and simple up to a few thousand items. Beyond that, send the filters to the API (the URL contract above already fits) and add pagination.

- *No debounce:* in-memory filtering is cheap. With server-side search, debounce the search input (about 300 ms) and cancel old requests (`AbortController`).
- *Derived on every render vs* `useMemo` *:* simpler and always correct; memoise only if profiling shows it's needed.
- *Invalid price range ignored (with an error) vs showing 0 results:* the user keeps seeing useful products and understands why the filter isn't applied.
- `replaceState` *vs* `pushState` *:* no history entry per keystroke, but the Back button doesn't step through filter changes. A router (React Router `useSearchParams`) would make this cleaner in a real app.
- `JSON.stringify` *to compare filters:* fine for a small flat object; for bigger objects compare field by field.

What to add with more time

- Tests: unit tests for `filterProducts` , `parsePriceRange` and the URL round-trip; React Testing Library for type → count updates, Reset, and error → Retry.
- Sorting (price low → high, rating), multi-select categories, and filter "chips" that can be removed one by one.
- Highlight the matched text in results; show the result count per category.
- Server-side filtering with debounce + `AbortController` + pagination / infinite scroll; a virtualised list for very large data.
- A skeleton loader instead of loading text, and keyboard shortcuts (`/` to focus search).

13. Counter questions the interviewer may ask

Requirements and design

Q1. Client-side or server-side filtering: how do you decide?

By data size and freshness. A few thousand items that rarely change → load once and filter on the client (instant). Large or fast-changing data → send the filters to the API (`?q=&category=&min=`) and paginate. The URL contract here works for both.

Q2. Should filters combine with AND or OR?

Across different filters: AND (category *and* price). Within one multi-select filter (two categories): OR. Ask, because it changes the predicate.

Q3. Why keep the filters in the URL?

Refresh keeps the view, users can share or bookmark a filtered link, and the Back button can work. The URL is the most durable, shareable state on the web.

State design

Q4. Why one `filters` object instead of six `useState` s?

One `updateFilter(name, value)` handler, a one-line reset (`setFilters(DEFAULT_FILTERS)`), easy comparison with defaults ("any filter active?") and easy URL serialisation.

Q5. Why are the price inputs stored as strings?

Inputs give strings, and "" must mean "no limit". `Number("")` is `0`, which would silently mean "max price 0". Keep the raw string and parse it in one place (`parsePriceRange`).

Q6. Why derive the category options from the data?

Hard-coded lists go stale: the data had "Appliances" but the dropdown didn't, so the Refrigerator was unreachable. `[...new Set(products.map(p => p.category))].sort()` always matches the data.

Q7. Is the URL a second source of truth?

No. It's read once on load (lazy initial state) and then only written from `filters`. One direction of sync avoids loops and conflicts.

Hard parts

Q8. What was the "All" rating bug in the first version?

The state started as `"All"`, but the option value was `"all"`. Selecting "All" set `"all"`; `"all" === "All"` is false, `Number("all")` is `NaN`, every comparison failed, and the list emptied. Fix: typed defaults (`minRating: 0`) and one sentinel value used everywhere.

Q9. Min price > max price. What should happen?

Don't silently show 0 results. Show an inline error and ignore the price filter until it's fixed, so the user still sees useful products and knows why.

Q10. Why the `ignore` flag in the fetch effect?

If the user clicks Retry twice, or the component unmounts, an older response could arrive after a newer one and overwrite it. The cleanup sets `ignore = true` so stale responses are dropped. `AbortController` also cancels the network request.

Q11. Why `replaceState` and not `pushState` for the URL?

Each keystroke in search would push a history entry, so Back would go back one letter at a time. `replaceState` updates the URL without adding history. For "Back undoes the last filter", push on discrete changes only (category, checkbox) and listen to `popstate`.

Implementation steps

Q12. How do you make search ignore case and spaces?

Normalise once: `search.trim().toLowerCase()`, and compare with `name.toLowerCase().includes(query)`. For accents ("café" vs "cafe"), use `normalize("NFD").replace(/\p{Diacritic}/gu, "")`.

Q13. How would you test `filterProducts`?

It's a pure function: give it a small product array and a filters object, assert the ids. Test each rule alone, the combinations, the boundaries (exactly 4.0 rating, exactly the min price), the invalid range, and the URL round trip `filtersFromURL(filtersToURL(f))`.

Q14. How do you handle garbage in the URL, e.g. `?rating=abc`?

Validate when reading: `Number("abc")` is `NaN`, which fails the `0..5` check, so it falls back to the default. Never trust URL values.

Performance and scale

Q15. What if there are 100,000 products?

Move filtering to the server with pagination. If it must stay on the client: debounce search, `useMemo` the filtered list, precompute lowercase names once, and virtualise the grid. For instant text search, use an index (e.g. a prefix trie or a search library like FlexSearch).

Q16. Should search be debounced here?

Not for in-memory filtering of 10 items: it's instant. Yes as soon as search calls an API (300-500 ms, plus `AbortController`), or if filtering becomes slow (also consider `useDeferredValue`).

Q17. How would you add sorting?

Add `sort` to the filters object (`price-asc`, `rating-desc`, ...), sort a *copy* after filtering (`[...list].sort(...)`, because `sort` mutates), and include it in the URL.

Trade-offs and "why not X?"

Q18. Why not `useMemo` for the filtered list?

It costs memory and complexity, and with 10 items there's nothing to save. Add it when profiling shows the filter is slow or the list feeds memoised children.

Q19. Why not React Router's `useSearchParams`?

In a real app with a router, yes: it handles the URL and re-renders for you. This app has no router, so `URLSearchParams` + `history.replaceState` keeps it dependency-free.

Q20. How would you show "number of results per category" next to each option?

Count with the *other* filters applied (not the category itself): `filterProducts(products, { ...filters, category: "all" })`, then group by category. This is called faceted search; on the server it's an aggregation.